Propuesta de desarrollo de software con GitHub y especificaciones para una empresa ecuatoriana

Kevin Jahir Coronel Gaona

Segundo integrante por registrar

Universidad Técnica de Machala

Ingeniería en Tecnología de la Información

Programación Móvil

Octavo semestre B

Docente por registrar

20 de octubre de 2026

Introducción

La empresa del caso presenta pérdida de versiones, desorganización del trabajo y errores que llegan a producción. El problema no se resuelve únicamente al instalar herramientas: hace falta acordar cómo se define una necesidad, quién revisa un cambio y qué comprobaciones autorizan su entrega. Este informe propone un proceso de desarrollo para un equipo pequeño y lo concreta en DevTask, una aplicación Angular con API Node.js y base de datos SQLite.

La propuesta utiliza Git para conservar cambios, GitHub para coordinar la revisión y GitHub Actions para ejecutar comprobaciones repetibles. Las especificaciones determinan el comportamiento esperado antes de implementar. La asistencia de inteligencia artificial se incorpora con revisión humana y evidencia de pruebas. El objetivo es reducir cambios sin trazabilidad y detectar defectos antes de integrar el código, sin atribuir mejoras cuantitativas que todavía no se han medido.

El alcance comprende investigación de siete bloques, una aplicación funcional, especificación, configuración de automatización y guías de trabajo. Las pruebas locales y la compilación corresponden a ejecuciones de esta preparación. La creación del repositorio remoto, las revisiones entre estudiantes y el uso efectivo de Copilot y Spec Kit requieren evidencias propias. La implementación se publicó en Kevin21JCG/Programacion, rama feat/gestion-tareas, mediante el Pull Request 2. La ejecución 39438736198 de GitHub Actions completó correctamente las pruebas, la compilación y la comprobación HTTP.

Control de versiones

Git permite conservar estados del proyecto mediante commits y relacionarlos con cambios concretos. Una rama permite preparar una funcionalidad sin alterar inmediatamente la versión compartida. En la propuesta, cada commit debe expresar una intención pequeña y

comprobable, como corregir una validación o incorporar una prueba. El equipo revisa el diff antes de guardar cambios y evita incluir dependencias instaladas, archivos de base de datos con información real y credenciales.

Entre las operaciones avanzadas, merge integra historias conservando su relación, mientras rebase reaplica commits sobre otra base y modifica sus identificadores. Reorganizar commits locales puede facilitar la revisión, pero reescribir una rama compartida sin coordinación obliga a otros a reconciliar historias diferentes (Chacon & Straub, 2014). Para deshacer un cambio ya compartido, la propuesta prefiere un nuevo commit de reversión y conserva el motivo del cambio.

Git Flow distingue ramas de desarrollo, funcionalidades, versiones y correcciones urgentes. Su estructura puede servir cuando se mantienen versiones publicadas en paralelo. Trunk Based Development favorece integrar cambios pequeños con frecuencia en una línea principal, con ramas breves cuando se necesitan. Driessen (2010) también advierte que su modelo no debe adoptarse de forma universal; el contexto de entrega importa. Para dos estudiantes y dos semanas, se propone main más ramas cortas y Pull Requests. Es una adaptación práctica de integración frecuente, no una implementación completa de Git Flow (Trunk Based Development, s. f.).

Trabajo colaborativo

GitHub complementa el repositorio con conversaciones, solicitudes de integración y revisión. Un Pull Request relaciona una propuesta de cambio con su diferencia de código y permite discutirla antes de integrar. La revisión debe considerar el requisito que originó el cambio y las pruebas asociadas, no limitarse a confirmar que el programa abre (GitHub, s. f.-e).

El flujo propuesto comienza con un Issue que contiene el problema y los criterios de aceptación. Su responsable crea una rama, implementa un cambio pequeño y abre un PR con instrucciones de prueba. El otro estudiante revisa el alcance, los errores y la claridad. Si encuentra un problema, solicita una corrección específica. Tras corregir y obtener comprobaciones satisfactorias, se integra el cambio. Esta revisión cruzada distribuye conocimiento y permite explicar quién tomó cada decisión.

En DevTask, una revisión útil consiste en comprobar que un título vacío se rechaza también en la API. Validarlo solamente en Angular deja abierta la posibilidad de enviar una petición HTTP incorrecta. Otro punto es comprobar que el texto con comillas se conserva sin romper la consulta. Estos criterios convierten el Code Review en una revisión del comportamiento y del riesgo, en lugar de una preferencia sobre estilo.

Desarrollo basado en especificaciones

Spec Driven Development organiza el trabajo alrededor de una descripción verificable del sistema. La especificación determina qué debe ocurrir y bajo qué restricciones. El plan selecciona arquitectura y tecnologías. Las tareas convierten esas decisiones en unidades de implementación. GitHub Spec Kit ofrece un conjunto de herramientas para apoyar esa secuencia y mantener los artefactos relacionados (GitHub, s. f.-f).

La especificación de DevTask define seis requisitos funcionales. RF01 establece límites para título, descripción y responsable, junto con los estados y prioridades válidos. RF02 exige buscar y filtrar. RF03 cubre edición y cambio de estado. RF04 exige confirmar la eliminación. RF05 exige persistencia en SQLite y consultas parametrizadas. RF06 presenta conteos globales. Los requisitos no funcionales añaden validación de servidor, mensajes de error, uso por teclado y una automatización reproducible.

Un criterio concreto de RF01 es que un título de menos de tres caracteres produzca HTTP 400 y no cree un registro. Para RF05, cerrar y abrir la conexión debe conservar la tarea. Estos criterios se traducen directamente en pruebas. El paquete contiene spec.md, plan.md y tasks.md escritos para esta versión, además de una constitución de principios. Son artefactos preparados manualmente: su presencia no demuestra que se haya ejecutado Spec Kit. La guía explica cómo usar la instalación oficial y registrar la evidencia correspondiente.

SDD aporta claridad cuando el equipo discute primero el resultado esperado. También introduce mantenimiento: si cambia la regla, deben actualizarse la especificación, la implementación y sus pruebas. Una especificación desactualizada puede dar una falsa sensación de control. La propuesta exige revisar estos tres elementos en el mismo PR para mantener su correspondencia.

DevOps y automatización

DevOps conecta el desarrollo con la operación mediante responsabilidad compartida y retroalimentación. En el caso estudiado, la primera mejora consiste en reemplazar comprobaciones informales por pasos repetibles. GitHub Actions permite definir workflows activados por eventos del repositorio. Para un proyecto Node.js, la instalación del entorno, las dependencias y las pruebas pueden expresarse como pasos del pipeline (GitHub, s. f.-d).

El workflow de calidad de DevTask instala las dependencias con npm ci, ejecuta cinco pruebas de negocio y API, compila Angular y levanta temporalmente la aplicación compilada. La prueba de humo consulta el HTML y la ruta de salud. Por último, genera un artefacto descargable asociado al commit. Si falla una prueba o la compilación, los pasos siguientes no completan la entrega. Subir un archivo de workflow no prueba que GitHub lo ejecutó: debe conservarse el enlace a una ejecución real.

La integración continua comprueba cada cambio. La entrega continua mantiene un resultado preparado para publicar con una decisión posterior. El despliegue continuo publicaría automáticamente los cambios aprobados por las verificaciones. En esta entrega se elige CI con despliegue efímero de prueba y un segundo workflow manual para un laboratorio. El segundo necesita un runner propio con Docker, conserva SQLite en un volumen y limita el acceso a localhost. No hay un servicio público de producción desplegado.

Una ampliación empresarial debe incluir autenticación, copias de seguridad consistentes, monitoreo y un procedimiento de recuperación. Un pipeline verde demuestra que se cumplieron las comprobaciones definidas, no que se descartó todo defecto posible. Por eso se conserva una revisión humana y se proponen métricas de fallos antes y después de la integración.

Inteligencia artificial aplicada al desarrollo

GitHub Copilot puede asistir con sugerencias y tareas de programación, pero sus resultados requieren evaluación. La documentación de uso responsable señala capacidades y limitaciones que el usuario debe considerar al revisar el resultado (GitHub, s. f.-c). En este proyecto, la IA se propone como apoyo para generar casos límite y explicar diferencias, manteniendo la decisión técnica en el equipo.

Un prompt útil incluye contexto, objetivo, restricciones y verificación. Por ejemplo: “Revisa RF01 y la función validar. Identifica una entrada que viole el contrato. Propón una prueba que la detecte antes de modificar código. Conserva SQLite y no cambies los límites del título”. El resultado esperado es una prueba pertinente y una explicación, no una reescritura extensa sin relación con el requisito.

El equipo debe registrar qué recomendación recibió, qué aceptó y qué corrigió. También debe evitar transmitir credenciales o información real de clientes. El código de esta entrega se

preparó con asistencia de ChatGPT y se verificó localmente; no se atribuye su autoría a Copilot ni se inventa una sesión con esa herramienta. La guía incluye prompts y un procedimiento para que los estudiantes obtengan y documenten una demostración real de Copilot.

Calidad de software y TDD

Test Driven Development plantea ciclos de prueba fallida, implementación suficiente y refactorización. La intención es que la prueba exprese una necesidad antes de escribir la solución y siga protegiendo el comportamiento cuando se reorganiza el código (Fowler, 2023). Tener pruebas automatizadas al final del trabajo es valioso, pero por sí solo no demuestra que se aplicó TDD durante todo el proceso.

En esta preparación, la suite se escribió antes de implementar los módulos y su primera ejecución falló al no encontrarlos. Después se implementó el contrato y las cinco pruebas pasaron. Esa evidencia representa una fase roja estructural y una fase verde. No se presenta como un historial completo de ciclos TDD por cada funcionalidad. Para una demostración más precisa, la guía propone acordar una regla adicional, escribir primero la prueba que falle por esa regla y registrar el cambio mínimo que la hace pasar.

La suite verifica límites y tipos de entrada; edición y eliminación; persistencia después de cerrar SQLite; conservación de texto con comillas y apariencia de SQL; y un ciclo HTTP que comprueba creación, lectura, actualización, errores y borrado. La compilación Angular usa tipado y plantillas estrictas. La prueba de humo complementa la suite al comprobar el servicio compilado. Los filtros visuales, los conteos y el uso por teclado también tienen una guía de verificación manual.

Gestión del proyecto

Los Issues describen unidades de trabajo y sus criterios. Projects permite organizarlos en vistas como un tablero, vinculando la información del trabajo con los registros de GitHub. Los milestones agrupan Issues y PR relacionados con una entrega (GitHub, s. f.-a, s. f.-b). La propuesta utiliza estas funciones para que cada decisión tenga un lugar y una persona responsable.

Se plantean tres hitos: especificación, implementación y validación. El tablero tendrá Pendiente, En curso, En revisión y Completada. Cada integrante mantendrá una tarea activa para reducir cambios simultáneos. Kevin y el segundo integrante, todavía por registrar, alternarán implementación y revisión. El plan de dos semanas incluido en el paquete es una propuesta de organización y no una bitácora de trabajo ya realizado.

Una tarea estará terminada cuando cumpla su aceptación, tenga verificación pertinente, documentación actualizada y revisión cruzada. Para evaluar el proceso se propone registrar el tiempo desde que inicia una tarea hasta su integración, los fallos de pipeline y los defectos detectados en revisión. Sin medición previa no corresponde afirmar que la propuesta reduce errores en un porcentaje específico.

Propuesta de solución para la empresa

Se propone iniciar con un proyecto piloto de alcance pequeño. DevTask representa ese piloto: un tablero de tareas desarrollado con Angular 20.3, TypeScript, una API Node.js y SQLite. La versión se ejecuta con Node.js 24 para mantener un entorno coherente con las herramientas seleccionadas. La documentación oficial de Angular relaciona versiones del framework, TypeScript y Node.js; esa compatibilidad debe revisarse antes de actualizar dependencias (Angular, s. f.).

El navegador recoge los datos y consulta la API. El servidor valida las reglas, ejecuta sentencias parametrizadas y guarda registros en SQLite. Angular no abre directamente la base de datos. Los campos incluyen identificador, título, descripción, responsable, prioridad, estado y fechas. El mismo servidor entrega el frontend compilado y la API cuando se ejecuta en modo local de entrega, evitando configurar dos orígenes diferentes.

SQLite se eligió por la facilidad de instalación y la posibilidad de demostrar persistencia en una computadora de clase. La aplicación no incluye login ni control de acceso y escucha en localhost de manera predeterminada. Tampoco gestiona conflictos de edición: prevalece la última escritura. Antes de usarla en una empresa real se deben acordar usuarios, permisos, respaldo y concurrencia. Estas restricciones son parte del alcance del piloto.

La implantación sugerida comienza con definir las reglas y publicar el repositorio. Luego, ambos estudiantes aplican el flujo de ramas y revisión a una mejora concreta. Al final se revisan los resultados del pipeline y se ensaya la recuperación de un cambio incorrecto. El criterio de éxito inicial es que una funcionalidad pueda relacionarse con su especificación, Issue, PR, prueba y evidencia de ejecución. La utilidad de cada herramienta se juzga por esa trazabilidad.

Análisis crítico

La principal ventaja del modelo es que hace visibles decisiones que antes dependían de mensajes informales. Su costo es el tiempo de mantenimiento y revisión. Para un equipo de dos personas, una estructura extensa de ramas y aprobaciones puede retrasar el trabajo sin mejorar su calidad. Por eso se eligen ramas breves, PR acotados y pruebas que cubran reglas relevantes, en lugar de intentar reproducir todos los procesos de una empresa grande.

También existe el riesgo de cumplir la rúbrica solo de forma aparente. Un tablero vacío no prueba organización, una respuesta de IA no prueba comprensión y un pipeline que no

verifica el comportamiento puede dejar pasar defectos. La propuesta exige evidencias observables y explica lo que cada una demuestra. El informe, el código y las guías son entregables preparados; las acciones en GitHub y la participación de ambos estudiantes deben comprobarse con registros reales.

Reflexión final

El problema que resuelve Git

Git permite identificar qué cambió y conservar una historia recuperable. En un desarrollo real facilita comparar versiones y coordinar cambios. Su beneficio depende de realizar commits comprensibles y sincronizar el trabajo; no reemplaza la comunicación del equipo ni una política de respaldo.

La importancia de DevOps

DevOps aporta una forma de comprobar y entregar cambios de manera repetible. En esta práctica reduce la dependencia de la frase “en mi computadora funciona”, porque el pipeline vuelve a instalar, probar y compilar en otro entorno. Aun así, el funcionamiento en producción requiere monitoreo y respuesta a incidentes.

El cambio que introduce la IA

La IA puede acelerar la elaboración de alternativas y pruebas, pero desplaza parte del esfuerzo hacia formular requisitos y revisar resultados. El programador debe explicar el código, detectar errores de la sugerencia y confirmar su comportamiento con pruebas. La velocidad de generación no sustituye esa responsabilidad.

La herramienta más útil para esta propuesta

GitHub Actions ofrece una comprobación común para todo el equipo y hace visibles los fallos antes de integrar. Por ello se considera la herramienta de mayor utilidad operativa dentro de la propuesta. Esta valoración se basa en su función dentro del diseño, no en una comparación experimental de productividad ya realizada.

Conclusiones

El caso requiere un proceso compartido, además de herramientas. Git conserva la evolución del código, GitHub organiza la revisión y Actions automatiza verificaciones. Las especificaciones permiten decidir qué debe demostrar cada cambio y las pruebas comprueban una parte de ese contrato.

DevTask materializa la propuesta con un frontend Angular y una base SQLite accesible mediante API. La validación local confirma los casos automatizados y la compilación, pero no reemplaza las evidencias de trabajo colaborativo. El repositorio, la rama de implementación, el PR y una ejecución correcta del pipeline ya cuentan con enlaces verificables. Sigue pendiente la revisión cruzada del otro integrante.

Para ampliar el piloto se recomienda medir primero los resultados, añadir controles de acceso y definir respaldos y recuperación. La IA debe utilizarse con prompts acotados y revisión documentada. El segundo integrante debe incorporarse a la portada y a la bitácora únicamente con su identidad y contribución reales.

Referencias

Angular. (s. f.). *Version compatibility*. Angular. https://angular.dev/reference/versions

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/en/v2/Git-Branching-Rebasing

Driessen, V. (2010, 5 de enero). *A successful Git branching model*. nvie.com. https://nvie.com/posts/a-successful-git-branching-model/

Fowler, M. (2023, 11 de diciembre). *Test driven development*. https://martinfowler.com/bliki/TestDrivenDevelopment.html

GitHub. (s. f.-a). *About milestones*. GitHub Docs. https://docs.github.com/issues/using-labels-and-milestones-to-track-work/about-milestones

GitHub. (s. f.-b). *About Projects*. GitHub Docs. https://docs.github.com/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects

GitHub. (s. f.-c). *Application card GitHub Copilot Agents*. GitHub Docs. https://docs.github.com/en/copilot/responsible-use/agents

GitHub. (s. f.-d). *Building and testing Node.js*. GitHub Docs. https://docs.github.com/en/actions/tutorials/build-and-test-code/nodejs

GitHub. (s. f.-e). *Reviewing proposed changes in a pull request*. GitHub Docs. https://docs.github.com/en/pull-requests/how-tos/review-pull-requests/reviewing-proposed-changes-in-a-pull-request

GitHub. (s. f.-f). *Spec Kit* [Software]. https://github.com/github/spec-kit

Trunk Based Development. (s. f.). *Introduction*. https://trunkbaseddevelopment.com/

Anexo de verificación y evidencias pendientes

Resultados locales de esta preparación: cinco pruebas automatizadas aprobadas, compilación Angular correcta y prueba HTTP del despliegue local correcta. Los registros de consola se adjuntan en documentacion/evidencias-locales. La prueba roja inicial falló por módulos todavía no implementados. No demuestra un defecto de negocio ni una revisión del compañero.

Evidencia real en GitHub: repositorio https://github.com/Kevin21JCG/Programacion; rama feat/gestion-tareas; Issue 1 de implementación; Pull Request https://github.com/Kevin21JCG/Programacion/pull/2; ejecución correcta https://github.com/Kevin21JCG/Programacion/actions/runs/39438736198. El PR incluye una revisión técnica asistida, identificada como tal. Pendiente: revisión cruzada del compañero. El tablero Projects y los milestones descritos son propuestas de gestión todavía no creadas. Pendiente en el equipo: registrar al segundo integrante y al docente, realizar y documentar una sesión con Copilot y ejecutar Spec Kit si se desea acreditar su uso práctico. No se incluyen capturas ni enlaces inventados.

Para la demostración, ejecutar npm run instalar y npm run demo dentro de DevTask, luego npm run dev y abrir localhost:4200. Crear una tarea, editarla, buscarla y comprobar su permanencia después de reiniciar. Finalmente ejecutar npm test, npm run build y npm run humo. La guía FLUJO_GITHUB.md conserva el procedimiento de publicación y el archivo EVIDENCIAS_GITHUB.md registra el estado actual y los enlaces reales.